KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH,
Narayanguda, Hyderabad – 500029

DEPARTMENT
OF
COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES
LAB RECORD

B.Tech. II YEAR II SEM (KR24)
ACADEMIC YEAR 2025-26

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(Approved by AICTE & Govt of T.S. and Affiliated to JNTUH)
NARAYANAGUDA, HYDERABAD - 500 029.

Certificate

This is to certify that the following is a Bonafide Record of the practical work done by _____

bearing Roll No _____ of _____ Branch of _____ year B.Tech Course in the _____ Laboratory During the Academic year _____ & _____ under our supervision.

Number of experiments completed: _____

Signature of Staff Member Incharge

Signature of Head of the Dept.

Signature of Internal Examiner

Signature of External Examiner

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

Daily Laboratory Assessment Sheet

Name of the Lab: Name of the Student:

Class: HT.No:

S.No.	Name of the Experiment	Date	Observation Marks	Record Marks	Viva Voce Marks	Total Marks	Signature of Faculty
	TOTAL						

INDEX

S.NO	CONTENTS	PAGE NO
I	Vision/Mission /PEOs/POs/PSOs	i
II	Syllabus	vii
III	Courseoutcomes, C0-PO Mapping, CO-PSO Mapping	ix
Exp No:	**List of Experiments**	
1	Write a program to create, insert an element, delete an element, traverse the linked list and find length of a singly linked list.	1
2	Write a program to create, insert an element, delete an element, traverse a doubly linked list and also traverse the doubly linked list in reverse order.	5
3	Write a program to create, insert an element, delete an element, traverse a circular linked list.	9
4	Write a program to implement stack operations using arrays and singly linked lists.	13
5	Write a program to convert an infix expression to postfix expression.	19
6	Write a program to evaluate a postfix expression.	22
7	Write a program to check whether an expression is having balanced parenthesis or not.	25
8	Write a program using stacks to check whether a String is a Palindrome or not.	28
9	Write a program to implement Queue operations using Array and Singly Linked List.	30
10	Write a program to implement Circular Queue operations using Array and Singly Linked List.	37
11	Write a program to implement a queue using two stacks.	43
12	Write a program to check if a queue can be sorted into another queue using a stack.	46
13	Write a program that implements the following sorting methods to sort a given list of integers in ascending order using merge sort and quick sort methods.	49
14	Write a program that implements Binary tree operations using arrays and doubly linked list.	53
15	Implement, for a binary tree, the inorder, preorder, postorder and level order traversal methods.	58
16	Write a program to find the second minimum node in a Binary tree.	63
17	Write a program to insert a node, delete a node and search for a node in a Binary Search Trees.	67
18	Write a program to create a graph using arrays and linked lists.	71
19	Implement the Depth First Search(DFS) and Breadth First Search(BFS) graph traversal methods.	75

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Narayanguda, Hyderabad – 500029

Department of Computer Science & Engineering

Vision of the Institution:

To be the fountain head of latest technologies, producing highly skilled, globally competent engineers.

Mission of the Institution:

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi modal platforms and prepare students to become successful professionals.
- To establish Industry Institute Interaction to make students ready for the industry.
- To provide exposure to students on latest hardware and software tools.
- To promote research-based projects/activities in the emerging areas of technology convergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students

Department of Computer Science & Engineering

Vision of the Department:

To be among the region's premier teaching and research Computer Science and Engineering department producing globally competent and socially responsible graduates in the most conducive academic environment.

Mission of the Department:

- To provide faculty with state of the art facilities for continuous professional development and research, both in foundational aspects and of relevance to emerging computing trends.
- To impart skills that transform students to develop technical solutions for societal needs and inculcate entrepreneurial talents.
- To inculcate an ability in students to pursue the advancement of knowledge in various specializations of Computer Science and Engineering and make them industry-ready.
- To engage in collaborative research with academia and industry and generate adequate resources for research activities for seamless transfer of knowledge resulting in sponsored projects and consultancy.
- To cultivate responsibility through sharing of knowledge and innovative computing solutions that benefits the society-at-large.
- To collaborate with academia, industry and community to set high standards in academic excellence and in fulfilling societal responsibilities.

PROGRAM OUTCOMES (POs)

PO1: Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6: The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOs)

PSO1: An ability to analyze the common business functions to design and develop appropriate Computer Science solutions for social upliftment.

PSO2: Shall have expertise on the evolving technologies like Python, Machine Learning, Deep Learning, Internet of Things (IOT), Data Science, Full stack development, Social Networks, Cyber Security, Big Data, Mobile Apps, CRM, ERP etc.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Graduates will endeavor to excel in their chosen careers as professionals, researchers and entrepreneurs on a global platform.

PEO2: Graduates will demonstrate the ability to solve challenges in the fields of Engineering and Technology simultaneously catering to societal needs.

PEO3: Graduates will strive to improve their learning curve by practicing Continuing Professional Development (CPD).

PEO4: Graduates will, at all times, adopt a professional demeanor by communicating effectively, working collaboratively, and maintaining the ethics & core values as befitting their education in interdisciplinary and emerging fields.

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTE)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

B.Tech. in COMPUTER SCIENCE AND ENGINEERING (AI & ML)
II Year II Semester Course Syllabus (KR23)
DATA STRUCTURES LAB (23CC406PC)
Common for CSE, CSE (DATA SCIENCE), CSE (AI&ML), IT

Prerequisites/ Corequisites:

	L	T	P	C
1. 23CS103ES - Programming for Problem Solving Course	0	0	2	1

Course Objectives: The course will help to

1. Introduce various concepts of C++ programming language.
2. Understand data structures such as stacks and queues.
3. Explore trees and graphs.
4. Introduce searching and sorting algorithms.
5. Implement Data Structures in real world problems.

Course Outcomes: After learning the concepts of this course, student is able to

1. Design and implement C++ programs for computing real life applications.
2. Understand basic elements of control statements, arrays, functions, pointers and strings, and data. structures like stacks, queues, and linked lists.
3. Implement searching and sorting algorithms.
4. Select appropriate Data Structure to solve the problems.
5. Test and debug the application.

List of Programs:

1. Write a program to create, insert an element, delete an element, traverse the linked list and find length of a singly linked list.
2. Write a program to create, insert an element, delete an element, traverse a doubly linked list and also traverse the doubly linked list in reverse order.
3. Write a program to create, insert an element, delete an element, traverse a circular linked list.
4. Write a program to implement stack operations using arrays and singly linked lists.
5. Write a program to convert an infix expression to postfix expression.
6. Write a program to evaluate a postfix expression.

7. Write a program to check whether an expression is having balanced parenthesis or not.
8. Write a program using stacks to check whether a String is a Palindrome or not.
9. Write a program to implement Queue operations using Arrays and singly linked lists.
10. Write a program to implement Circular Queue operations using Arrays and singly linked lists.
11. Write a program to implement a queue using two stacks.
12. Write a program to check if a queue can be sorted into another queue using a stack.
13. Write a program that implements the following sorting methods to sort a given list of integers in ascending order using merge sort and quick sort methods.
14. Write a program that implements Binary tree operations using arrays and doubly linked list.
15. Implement, for a binary tree, the inorder, preorder, postorder and level order traversal methods.
16. Write a program to find the second minimum node in a Binary tree.
17. Write a program to insert a node, delete a node and search for a node in a Binary Search Trees.
18. Write a program to create a graph using arrays and linked lists.
19. Implement the depth first search and breadth first search graph traversal methods.

TEXTBOOKS:

1. Data Structures Through C++ - Yashavant Kanetkar, 4th Edition, BPB Publications, 2022.
2. Data structures using C++- D. S. Malik, 2nd Edition, Cengage learning, 2009.
3. The Complete Reference C++- Herbert Schildt, 4th Edition, Tata Mc Graw Hill, 2017.

REFERENCE BOOKS:

1. Data Structures and Algorithm Analysis in C++, 4th Edition, Weiss Mark Allen, Pearson Education · 2014
2. The C++ Programming Language, 4th Edition, B.Stroutstrup, Pearson Education, 2013.

Course Outcomes and CO-PO/PSO Mapping

At the end of the course a student will be able to

	Course Outcome (CO)
CO1	Design and implement C++ programs for computing real life applications.
CO2	Understand basic elements of control statements, arrays, functions, pointers and strings, and data. structures like stacks, queues, and linked lists.
CO3	Implement searching and sorting algorithms.
CO4	Select appropriate Data Structure to solve the problems.
CO5	Test and debug the application.

CO-PO- PSO MAPPING:

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	2	3	1	2	-	-	-	-	1	-	1	3	2
CO2	3	2	2	-	2	-	-	-	-	-	-	1	2	3
CO3	3	3	2	1	2	-	-	-	-	-	-	1	2	3
CO4	3	3	3	2	2	-	-	-	-	-	-	1	3	3
CO5	2	3	2	3	2	-	-	-	1	1	-	2	2	2

1: Low 2:Medium 3: High

Singly Linked List Operations

Aim

Write a program to create, insert an element, delete an element, traverse the linked list, and find the length of a singly linked list.

Algorithm

1. Create nodes dynamically.
2. Insert nodes into the linked list.
3. Delete a specified node.
4. Traverse and display all nodes.
5. Count the number of nodes.

Sample Input

Enter number of nodes: 4
Enter elements: 10 20 30 40
Insert element: 25
Delete element: 20

Sample Output

Linked List:
10 25 30 40
Length of Linked List = 4

Solution

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    // Constructor
    Node(int val) {
        data = val;
        next = NULL;
    }
};

class SinglyLinkedList {
    Node* head;

public:
    // Constructor
    SinglyLinkedList() {

        head = NULL;
    }

    // Insert at end
    void insert(int val) {
        Node* temp = new Node(val);

        if (head == NULL) {
            head = temp;
            return;
        }

        Node* p = head;

        while (p->next != NULL)
            p = p->next;

        p->next = temp;
    }

    // Delete a node
    void deleteNode(int val) {
        if (head == NULL)
            return;

        // Delete first node
        if (head->data == val) {
            Node* temp = head;
```

```
            head = head->next;
            delete temp;
            return;
        }

        Node* p = head;

        while (p->next != NULL && p->next->data != val)
            p = p->next;

        if (p->next == NULL)
            return;

        Node* temp = p->next;
        p->next = temp->next;
        delete temp;
    }

    // Display linked list
    void display() {
        Node* temp = head;

        while (temp != NULL) {
            cout << temp->data << " ";
            temp = temp->next;
        }

        cout << endl;
    }

    // Find length
    int length() {
        int count = 0;
        Node* temp = head;

        while (temp != NULL) {
            count++;
            temp = temp->next;
        }

        return count;
    }
};

int main() {
    SinglyLinkedList s;

    int n, x, insertElement, deleteElement;

    cout << "Enter number of nodes: ";
    cin >> n;

    cout << "Enter elements: ";
```

```
    for (int i = 0; i < n; i++) {
        cin >> x;
        s.insert(x);
    }

    cout << "Insert element: ";
    cin >> insertElement;
    s.insert(insertElement);

    cout << "Delete element: ";
    cin >> deleteElement;
    s.deleteNode(deleteElement);

    cout << "\nLinked List:\n";
    s.display();

    cout << "Length of Linked List = " << s.length();

    return 0;
}
```

Doubly Linked List Operations

Aim

To implement insertion, deletion, forward traversal, and reverse traversal in a doubly linked list.

Algorithm:

1. Define node with prev, data, next.
2. Insert/delete by adjusting both prev and next pointers.
3. Forward traversal: head to NULL.
4. Reverse traversal: tail to NULL.

Sample Input

Enter number of elements: 4
Enter elements: 11 22 33 44
Insert element: 55
Delete element: 22

Sample Output

Forward Traversal:
11 33 44 55
Reverse Traversal:
55 44 33 11

Solution

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* prev;
    Node* next;

    // Constructor
    Node(int val) {
        data = val;
        prev = NULL;
        next = NULL;
    }
};

class DoublyLinkedList {
    Node* head;

public:
    // Constructor
    DoublyLinkedList() {
        head = NULL;    }

    // Insert at end
    void insert(int val) {
        Node* temp = new Node(val);

        if (head == NULL) {
            head = temp;
            return;
        }

        Node* p = head;

        while (p->next != NULL)
            p = p->next;

        p->next = temp;
        temp->prev = p;
    }

    // Delete node
    void deleteNode(int val) {
        if (head == NULL)
            return;

        Node* temp = head;
```

```
    // Delete first node
    if (head->data == val) {
        head = head->next;

        if (head != NULL)
            head->prev = NULL;

        delete temp;
        return;
    }

    while (temp != NULL && temp->data != val)
        temp = temp->next;

    if (temp == NULL)
        return;

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    if (temp->prev != NULL)
        temp->prev->next = temp->next;

    delete temp;
}

// Forward Traversal
void displayForward() {
    Node* temp = head;

    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
// Reverse Traversal
void displayReverse() {
    if (head == NULL)
        return;

    Node* temp = head;

    // Move to last node
    while (temp->next != NULL)
        temp = temp->next;
    // Traverse backward
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->prev;
    }
    cout << endl;
}
```

```
};
int main() {
    DoublyLinkedList d;
    int n, x, insertElement, deleteElement;
    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> x;
        d.insert(x);
    }
    cout << "Insert element: ";
    cin >> insertElement;
    d.insert(insertElement);
    cout << "Delete element: ";
    cin >> deleteElement;
    d.deleteNode(deleteElement);
    cout << "\nForward Traversal:\n";
    d.displayForward();
    cout << "\nReverse Traversal:\n";
    d.displayReverse();

    return 0;}
```

Circular Linked List Operations

Aim

To implement insertion, deletion, and traversal operations in a circular linked list.

Algorithm:

1. Last node's next points to head.
2. Insert/delete similar to singly linked but maintain circularity.
3. Traversal stops when we reach head again.

Sample Input

Enter number of elements: 4
Enter elements: 1 2 3 4
Insert element: 5
Delete element: 2

Sample Output

Circular Linked List:
1 3 4 5

Solution

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    // Constructor
    Node(int val) {
        data = val;
        next = NULL;
    }
};

class CircularLinkedList {
    Node* tail;

public:
    // Constructor
    CircularLinkedList() {
        tail = NULL;
    }

    // Insert at end
    void insert(int val) {
        Node* temp = new Node(val);

        // First node
        if (tail == NULL) {
            tail = temp;
            tail->next = tail;
            return;
        }

        temp->next = tail->next;
        tail->next = temp;
        tail = temp;
    }

    // Delete node
    void deleteNode(int val) {
        if (tail == NULL)
            return;

        Node* curr = tail->next;
        Node* prev = tail;

        // Single node case
        if (curr == tail && curr->data == val) {
```

```
            delete curr;
            tail = NULL;
            return;
        }

        do {
            if (curr->data == val) {

                prev->next = curr->next;

                // If deleting tail
                if (curr == tail)
                    tail = prev;

                delete curr;
                return;
            }

            prev = curr;
            curr = curr->next;

        } while (curr != tail->next);
    }

    // Display circular linked list
    void display() {
        if (tail == NULL)
            return;

        Node* temp = tail->next;

        do {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp != tail->next);

        cout << endl;
    }
};

int main() {
    CircularLinkedList c;

    int n, x, insertElement, deleteElement;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> x;
        c.insert(x);
    }
```

```
    cout << "Insert element: ";
    cin >> insertElement;
    c.insert(insertElement);

    cout << "Delete element: ";
    cin >> deleteElement;
    c.deleteNode(deleteElement);

    cout << "\nCircular Linked List:\n";
    c.display();

    return 0;
}
```

Stack using Arrays and Linked Lists

Aim

Write a program to implement stack operations (push, pop, peek, isEmpty) using arrays and singly linked lists.

Stack using Arrays

Algorithm

1. Initialize `top = -1`.
2. Push:
    - Increment top.
    - Insert element at `arr[top]`.
3. Pop:
    - Remove element from `arr[top]`.
    - Decrement top.
4. Peek:
    - Return `arr[top]`.
5. Display:
    - Print elements from top to bottom.

Sample Input

Push: 10 20 30
Pop one element

Sample Output

Stack Elements:
20 10
Top Element: 20

Solution

```
#include <iostream>
using namespace std;

class Stack {
   int arr[100];
   int top;

public:
   // Constructor
   Stack() {
      top = -1;
   }

   // Push operation
   void push(int val) {
      if (top == 99) {
         cout << "Stack Overflow\n";
         return;
      }

      arr[++top] = val;
   }

   // Pop operation
   void pop() {
      if (isEmpty()) {
         cout << "Stack Underflow\n";
         return;
      }

      cout << "Popped Element: " << arr[top] << endl;
      top--;
   }

   // Peek operation
   void peek() {
      if (isEmpty()) {
         cout << "Stack is Empty\n";
         return;
      }

      cout << "Top Element: " << arr[top] << endl;
   }

   // isEmpty
   bool isEmpty() {
      return top == -1;
   }

   // Display stack
```

```
    void display() {
        if (isEmpty()) {
            cout << "Stack is Empty\n";
            return;
        }

        cout << "Stack Elements:\n";

        for (int i = top; i >= 0; i--)
            cout << arr[i] << " ";

        cout << endl;
    }
};

int main() {
    Stack s;

    s.push(10);
    s.push(20);
    s.push(30);

    s.pop();

    s.display();

    s.peek();

    return 0;
}
```

Stack using Singly Linked List

Aim

To implement stack operations using singly linked lists.

Operations

- Push
- Pop
- Peek
- Display
- isEmpty

Algorithm

- Push:Insert node at beginning.
- Pop:Delete node from beginning.
- Peek:Display head node data.
- Display:Traverse linked list.

Sample Input

Push: 10 20 30
Pop one element

Sample Output

Stack Elements:
20 10
Top Element: 20

Solution

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    // Constructor
    Node(int val) {
        data = val;
        next = NULL;
    }
};

class Stack {
    Node* top;

public:
    // Constructor
    Stack() {
        top = NULL;
    }

    // Push operation
    void push(int val) {
        Node* temp = new Node(val);

        temp->next = top;
        top = temp;
    }

    // Pop operation
    void pop() {
        if (isEmpty()) {
            cout << "Stack Underflow\n";
            return;
        }

        Node* temp = top;

        cout << "Popped Element: " << top->data << endl;

        top = top->next;

        delete temp;
    }

    // Peek operation
    void peek() {
```

```
        if (isEmpty()) {
            cout << "Stack is Empty\n";
            return;
        }

        cout << "Top Element: " << top->data << endl;
    }

    // isEmpty
    bool isEmpty() {
        return top == NULL;
    }

    // Display stack
    void display() {
        if (isEmpty()) {
            cout << "Stack is Empty\n";
            return;
        }

        cout << "Stack Elements:\n";

        Node* temp = top;

        while (temp != NULL) {
            cout << temp->data << " ";
            temp = temp->next;
        }

        cout << endl;
    }
};

int main() {
    Stack s;

    s.push(10);
    s.push(20);
    s.push(30);

    s.pop();

    s.display();

    s.peek();

    return 0;
}
```

Infix to Postfix Conversion

Aim

Write a program to convert an infix expression to postfix expression using stack. .

Algorithm:

1. Scan the infix expression from left to right.
2. If the symbol is an operand, add it to the postfix expression.
3. If the symbol is '(', push it onto the stack.
4. If the symbol is ')', pop and output from the stack until '(' is found.
5. If the symbol is an operator:
6. Pop operators with higher or equal precedence from the stack and add to output.
7. Push the current operator onto the stack.
8. Repeat until the expression ends.
9. Pop all remaining operators from the stack and append to postfix expression.

Sample Input

(A+B)*C-D

Sample Output

Postfix Expression: AB+C*D-

Solution

```
#include <iostream>
#include <stack>
using namespace std;

// Function to check precedence
int precedence(char op) {
   if (op == '+' || op == '-')
      return 1;

   if (op == '*' || op == '/')
      return 2;

   if (op == '^')
      return 3;

   return 0;
}

int main() {
   string infix, postfix = "";

   stack<char> st;

   cout << "Enter Infix Expression: ";
   cin >> infix;

   for (int i = 0; i < infix.length(); i++) {

      char ch = infix[i];

      // Operand
      if (isalnum(ch)) {
         postfix += ch;
      }

      // Left parenthesis
      else if (ch == '(') {
         st.push(ch);
      }

      // Right parenthesis
      else if (ch == ')') {

         while (!st.empty() && st.top() != '(') {
            postfix += st.top();
            st.pop();
         }

         st.pop(); // remove '('
      }
```

```
        // Operator
        else {

            while (!st.empty() &&
                precedence(st.top()) >= precedence(ch)) {

                postfix += st.top();
                st.pop();
            }

            st.push(ch);
        }
    }

    // Pop remaining operators
    while (!st.empty()) {
        postfix += st.top();
        st.pop();
    }

    cout << "Postfix Expression: " << postfix;

    return 0;
}
```

Postfix Expression Evaluation

Aim

Write a program to evaluate a postfix expression using stack.

Algorithm:

1. Scan the postfix expression from left to right.
2. If the symbol is an operand:
3. Push it onto the stack.
4. If the symbol is an operator:
5. Pop two operands from the stack.
6. Apply the operator.
7. Push the result back onto the stack.
8. Repeat until the expression ends.
9. The final value in the stack is the result.

Sample Input

Enter Postfix Expression: 23*54*+9-

Sample Output

Result = 17

Solution

```
#include <iostream>
#include <stack>
using namespace std;

int main() {

    string postfix;

    stack<int> st;

    cout << "Enter Postfix Expression: ";
    cin >> postfix;

    for (int i = 0; i < postfix.length(); i++) {

        char ch = postfix[i];

        // Operand
        if (isdigit(ch)) {
            st.push(ch - '0');
        }

        // Operator
        else {

            int b = st.top();
            st.pop();

            int a = st.top();
            st.pop();

            switch (ch) {

                case '+':
                    st.push(a + b);
                    break;

                case '-':
                    st.push(a - b);
                    break;

                case '*':
                    st.push(a * b);
                    break;

                case '/':
                    st.push(a / b);
                    break;

                case '^': {
```

```
                int result = 1;

                for (int j = 0; j < b; j++)
                    result *= a;

                st.push(result);
                break;
            }
        }
    }
}

cout << "Result = " << st.top();

return 0;
}
```

Balanced Parenthesis Checking

Aim

Write a program to check whether an expression has balanced parentheses using stack.

Algorithm:

1. Scan the expression from left to right.
2. If an opening bracket (, {, [is found:
3. Push it onto the stack.
4. If a closing bracket), },] is found:
5. Check whether the stack is empty.
6. Pop from the stack and verify matching brackets.
7. If mismatch occurs or stack becomes empty unexpectedly:
8. Expression is unbalanced.
9. After scanning complete expression:
10. If stack is empty → Balanced
11. Otherwise → Unbalanced

Sample Input-1

Enter Expression: {[(a+b)*(c+d)]}

Sample Output-1

Balanced Expression

Sample Input-2

Enter Expression: {[(a+b])}

Sample Output-2

Unbalanced Expression

Solution

```
#include <iostream>
#include <stack>
using namespace std;

// Function to check matching brackets
bool isMatching(char open, char close) {

   if (open == '(' && close == ')')
      return true;

   if (open == '{' && close == '}')
      return true;

   if (open == '[' && close == ']')       return true;

   return false;
}

int main() {

   string exp;

   stack<char> st;

   bool balanced = true;

   cout << "Enter Expression: ";
   cin >> exp;

   for (int i = 0; i < exp.length(); i++) {

      char ch = exp[i];

      // Opening brackets
      if (ch == '(' || ch == '{' || ch == '[') {
         st.push(ch);
      }

      // Closing brackets
      else if (ch == ')' || ch == '}' || ch == ']') {

         if (st.empty()) {
            balanced = false;
            break;
         }

         char top = st.top();
         st.pop();

         if (!isMatching(top, ch)) {
```

```
                balanced = false;
                break;
            }
        }
    }

    // If stack still contains elements
    if (!st.empty())
        balanced = false;

    if (balanced)
        cout << "Balanced Expression";
    else
        cout << "Unbalanced Expression";

    return 0;
}
```

Palindrome Checking using Stack

Aim

Write a program using stacks to check whether a string is a palindrome.

Algorithm:

1. Read the input string.
2. Push all characters of the string onto the stack.
3. Traverse the original string from beginning.
4. Pop characters one by one from the stack.
5. Compare popped characters with original string characters.
6. If all characters match:String is a palindrome.
7. Otherwise:String is not a palindrome.

Sample Input-1

Enter string : madam

Sample Output-1

Palindrome

Sample Input-2

Enter string : kmit

Sample Output-2

Not a Palindrome

Solution

```
#include <iostream>
#include <stack>
using namespace std;

int main() {

   string str;

   stack<char> st;

   bool palindrome = true;

   cout << "Enter string : ";
   cin >> str;

   // Push all characters into stack
   for (int i = 0; i < str.length(); i++) {
      st.push(str[i]);
   }

   // Compare with original string
   for (int i = 0; i < str.length(); i++) {

      char ch = st.top();
      st.pop();

      if (str[i] != ch) {
         palindrome = false;
         break;
      }
   }

   if (palindrome)
      cout << "Palindrome";
   else
      cout << "Not a Palindrome";

   return 0;
}
```

Queue using Arrays and Linked Lists

Aim

Write a program to implement queue operations (enqueue, dequeue, front, isEmpty) using arrays and singly linked lists.

Queue using Arrays

Operations

- Enqueue
- Dequeue
- Front
- isEmpty
- Display

Algorithm

1. Initialize `front = 0` and `rear = -1`.
2. Enqueue:
    - Increment rear.
    - Insert element at `arr[rear]`.
3. Dequeue:
    - Remove element from `front`.
    - Increment front.
4. Front:
    - Display element at `arr[front]`.
5. isEmpty:
    - Queue is empty if `front > rear`.

Sample Input

Enter number of elements: 4
Enter queue elements: 10 20 30 40

Sample Output

Deleted Element: 10

Queue Elements:
20 30 40

Front Element: 20

Solution

```
#include <iostream>
using namespace std;

class Queue {
   int arr[100];
   int front, rear;

public:

   // Constructor
   Queue() {
      front = 0;
      rear = -1;
   }

   // Enqueue operation
   void enqueue(int val) {

      if (rear == 99) {
         cout << "Queue Overflow\n";
         return;
      }

      arr[++rear] = val;
   }

   // Dequeue operation
   void dequeue() {

      if (isEmpty()) {
         cout << "Queue Underflow\n";
         return;
      }

      cout << "Deleted Element: " << arr[front] << endl;
      front++;
   }

   // Front element
   void frontElement() {

      if (isEmpty()) {
         cout << "Queue is Empty\n";
         return;
      }

      cout << "Front Element: " << arr[front] << endl;
   }

   // isEmpty
```

```
    bool isEmpty() {
        return front > rear;
    }

    // Display queue
    void display() {

        if (isEmpty()) {
            cout << "Queue is Empty\n";
            return;
        }

        cout << "Queue Elements:\n";

        for (int i = front; i <= rear; i++)
            cout << arr[i] << " ";

        cout << endl;
    }
};

int main() {

    Queue q;
    int n, x;
    cout << "Enter number of elements: ";
    cin >> n;
    cout << "Enter queue elements: ";
    for (int i = 0; i < n; i++) {
        cin >> x;
        q.enqueue(x);
    }
    q.dequeue();
    cout << endl;
    q.display();
    q.frontElement();
    return 0;
}
```

Queue using Singly Linked List

Aim

To implement queue operations using singly linked list.

Sample Input

Enter number of elements: 4
Enter queue elements: 10 20 30 40

Sample Output

Deleted Element: 10

Queue Elements:
20 30 40

Front Element: 20

Solution

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    // Constructor
    Node(int val) {
        data = val;
        next = NULL;
    }
};

class Queue {

    Node* front;
    Node* rear;

public:

    // Constructor
    Queue() {
        front = rear = NULL;
    }

    // Enqueue operation
    void enqueue(int val) {

        Node* temp = new Node(val);

        if (rear == NULL) {
            front = rear = temp;
            return;
        }

        rear->next = temp;
        rear = temp;
    }

    // Dequeue operation
    void dequeue() {

        if (isEmpty()) {
            cout << "Queue Underflow\n";
            return;
        }

        Node* temp = front;
```

```
        cout << "Deleted Element: " << front->data << endl;

        front = front->next;

        if (front == NULL)
            rear = NULL;

        delete temp;
    }

    // Front element
    void frontElement() {

        if (isEmpty()) {
            cout << "Queue is Empty\n";
            return;
        }

        cout << "Front Element: " << front->data << endl;
    }

    // isEmpty
    bool isEmpty() {
        return front == NULL;
    }

    // Display queue
    void display() {

        if (isEmpty()) {
            cout << "Queue is Empty\n";
            return;
        }

        cout << "Queue Elements:\n";

        Node* temp = front;

        while (temp != NULL) {
            cout << temp->data << " ";
            temp = temp->next;
        }

        cout << endl;
    }
};

int main() {

    Queue q;

    int n, x;
```

```
    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter queue elements: ";

    for (int i = 0; i < n; i++) {
        cin >> x;
        q.enqueue(x);
    }

    q.dequeue();

    cout << endl;

    q.display();

    q.frontElement();

    return 0;
}
```

Circular Queue using Arrays and Linked Lists

Aim

Write a program to implement circular queue operations using arrays and singly linked lists.

Circular Queue using Arrays

Algorithm (Array):

- Use (rear+1) % MAX to wrap around.
- Full when (rear+1)%MAX == front.
- Empty when front == -1.

Sample Input

Enter number of elements to insert: 3
Insert elements: 5 10 15
Insert one more element: 20

Sample Output

Deleted Element: 5

Circular Queue:
10 15 20

Solution

```
#include <iostream>
using namespace std;

class CircularQueue {

    int arr[100];
    int front, rear;
    int MAX = 100;

public:

    // Constructor
    CircularQueue() {
        front = rear = -1;
    }

    // Insert operation
    void insert(int val) {

        // Queue Full
        if ((rear + 1) % MAX == front) {
            cout << "Queue Overflow\n";
            return;
        }

        // First element
        if (front == -1) {
            front = rear = 0;
        }

        else {
            rear = (rear + 1) % MAX;
        }

        arr[rear] = val;
    }

    // Delete operation
    void deleteElement() {

        if (front == -1) {
            cout << "Queue Underflow\n";
            return;
        }

        cout << "Deleted Element: " << arr[front] << endl;

        // Single element
        if (front == rear) {
            front = rear = -1;
```

```
        }

        else {
            front = (front + 1) % MAX;
        }
    }

    // Display queue
    void display() {

        if (front == -1) {
            cout << "Queue is Empty\n";
            return;
        }

        cout << "Circular Queue:\n";

        int i = front;

        while (i != rear) {
            cout << arr[i] << " ";
            i = (i + 1) % MAX;
        }

        cout << arr[rear] << endl;
    }
};

int main() {

    CircularQueue q;

    int n, x;

    cout << "Enter number of elements to insert: ";
    cin >> n;

    cout << "Insert elements: ";

    for (int i = 0; i < n; i++) {
        cin >> x;
        q.insert(x);
    }
    q.deleteElement();
    cout << "Insert one more element: ";
    cin >> x;
    q.insert(x);
    cout << endl;
    q.display();
    return 0;
}
```

Circular Queue using Singly Linked List

Aim

To implement circular queue operations using singly linked list.

Algorithm

1. Maintain rear pointer.
2. Last node points to front node.
3. Insert:
    - Add node after rear.
4. Delete:
    - Remove front node.
5. Traverse until starting node repeats.

Sample Input

Enter number of elements to insert: 3
Insert elements: 5 10 15
Insert one more element: 20

Sample Output

Deleted Element: 5

Circular Queue:
10 15 20

Solution

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    // Constructor
    Node(int val) {
        data = val;
        next = NULL;
    }
};

class CircularQueue {

    Node* rear;

public:

    // Constructor
    CircularQueue() {
        rear = NULL;
    }

    // Insert operation
    void insert(int val) {

        Node* temp = new Node(val);

        // First node
        if (rear == NULL) {
            rear = temp;
            rear->next = rear;
            return;
        }

        temp->next = rear->next;
        rear->next = temp;
        rear = temp;
    }

    // Delete operation
    void deleteElement() {

        if (rear == NULL) {
            cout << "Queue Underflow\n";
            return;
        }
```

```
        Node* front = rear->next;

        cout << "Deleted Element: " << front->data << endl;

        // Single node
        if (rear == front) {
            delete front;
            rear = NULL;
            return;
        }

        rear->next = front->next;
        delete front;
    }
    // Display queue
    void display() {
        if (rear == NULL) {
            cout << "Queue is Empty\n";
            return;
        }
        cout << "Circular Queue:\n";
        Node* temp = rear->next;
        do {
            cout << temp->data << " ";
            temp = temp->next;
        }
        while (temp != rear->next);
        cout << endl;
    }
};

int main() {
    CircularQueue q;
    int n, x;
    cout << "Enter number of elements to insert: ";
    cin >> n;
    cout << "Insert elements: ";
    for (int i = 0; i < n; i++) {
        cin >> x;
        q.insert(x);
    }
    q.deleteElement();
    cout << "Insert one more element: ";
    cin >> x;

    q.insert(x);
    cout << endl;
    q.display();
    return 0;
}
```

Queue using Two Stacks

Aim

Write a program to implement a queue using two stacks.

Algorithm

1. Enqueue Operation:
    - Push elements into `stack1`.
2. Dequeue Operation:
    - If `stack2` is empty:
        - Pop all elements from `stack1`
        - Push them into `stack2`
    - Pop element from `stack2`
3. Display Operation:
    - Elements in `stack2` are displayed first.
    - Then elements from `stack1` are displayed in reverse order.

Sample Input

Enter number of elements to enqueue: 3
Enter elements: 1 2 3

Sample Output

Deleted Element: 1

Queue Elements:
2 3

Solution

```
#include <iostream>
#include <stack>
using namespace std;

class Queue {

   stack<int> stack1, stack2;

public:

   // Enqueue operation
   void enqueue(int val) {
      stack1.push(val);
   }

   // Dequeue operation
   void dequeue() {

      if (isEmpty()) {
         cout << "Queue Underflow\n";
         return;
      }

      // Transfer elements if stack2 is empty
      if (stack2.empty()) {

         while (!stack1.empty()) {
            stack2.push(stack1.top());
            stack1.pop();
         }
      }

      cout << "Deleted Element: " << stack2.top() << endl;

      stack2.pop();
   }

   // Check if queue is empty
   bool isEmpty() {
      return stack1.empty() && stack2.empty();
   }

   // Display queue elements
   void display() {

      if (isEmpty()) {
         cout << "Queue is Empty\n";
         return;
      }
```

```
        cout << "Queue Elements:\n";

        // Temporary stacks for display
        stack<int> temp1 = stack1;
        stack<int> temp2 = stack2;

        // Display stack2 elements
        while (!temp2.empty()) {
            cout << temp2.top() << " ";
            temp2.pop();
        }

        // Store stack1 elements
        int arr[100];
        int i = 0;

        while (!temp1.empty()) {
            arr[i++] = temp1.top();
            temp1.pop();
        }

        // Print in reverse
        for (int j = i - 1; j >= 0; j--) {
            cout << arr[j] << " ";
        }

        cout << endl;
    }
};

int main() {

    Queue q;

    int n, x;

    cout << "Enter number of elements to enqueue: ";
    cin >> n;

    cout << "Enter elements: ";

    for (int i = 0; i < n; i++) {
        cin >> x;
        q.enqueue(x);
    }

    q.dequeue();
    cout << endl;
    q.display();
    return 0;
}
```

Queue Sorting using Stack

Aim

Write a program to check if a queue can be sorted into another queue using a stack.

Algorithm

1. Initialize `expected = 1`.
2. Traverse the queue:
    - If front element equals `expected`:
        - Dequeue it.
        - Increment `expected`.
    - Else if stack top equals `expected`:
        - Pop from stack.
        - Increment `expected`.
    - Else:
        - Push front element into stack.
3. After queue becomes empty:
    - Pop remaining elements from stack if they match `expected`.
4. If all elements are matched in sorted order:
    - Queue can be sorted.
5. Otherwise:
    - Queue cannot be sorted.

Sample Input

Enter number of elements: 5
Queue Elements:
5 1 2 3 4

Sample Output

Queue can be sorted

Sample Input

Enter number of elements: 5
Queue Elements:
5 1 2 4 3

Sample Output

Queue cannot be sorted

Solution

```
#include <iostream>
#include <queue>
#include <stack>
using namespace std;

bool canBeSorted(queue<int> q, int n) {

    stack<int> st;

    int expected = 1;

    while (!q.empty()) {

        // If front matches expected
        if (q.front() == expected) {
            q.pop();
            expected++;
        }

        // If stack top matches expected
        else if (!st.empty() && st.top() == expected) {
            st.pop();
            expected++;
        }

        // Push front into stack
        else {
            st.push(q.front());
            q.pop();
        }
    }

    // Check remaining stack
    while (!st.empty()) {

        if (st.top() == expected) {
            st.pop();
            expected++;
        }

        else {
            return false;
        }
    }

    return true;
}

int main() {
```

```
    queue<int> q;

    int n, x;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Queue Elements:\n";

    for (int i = 0; i < n; i++) {
        cin >> x;
        q.push(x);
    }

    if (canBeSorted(q, n))
        cout << "Queue can be sorted";
    else
        cout << "Queue cannot be sorted";

    return 0;
}
```

Merge Sort and Quick Sort

Aim

To sort a list of integers using Merge Sort and Quick Sort.

Merge Sort

Algorithm

1. Divide the array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves into a single sorted array.
4. Repeat until the entire array is sorted.

Sample Input

Enter number of elements: 6
Enter elements: 45 12 89 23 1 67

Sample Output

Sorted Array:
1 12 23 45 67 89

Solution

```
#include <iostream>
using namespace std;
// Merge function
void merge(int arr[], int low, int mid, int high) {
   int temp[100];
   int i = low;
   int j = mid + 1;
   int k = low;
   // Merge two sorted subarrays
   while (i <= mid && j <= high) {
      if (arr[i] < arr[j]) {
         temp[k++] = arr[i++];
      }
      else {
         temp[k++] = arr[j++];
      }
   }
   // Copy remaining elements
   while (i <= mid)
      temp[k++] = arr[i++];
   while (j <= high)
      temp[k++] = arr[j++];
   // Copy back to original array
   for (int p = low; p <= high; p++) {
      arr[p] = temp[p];
   }
}
// Merge Sort function
void mergeSort(int arr[], int low, int high) {
   if (low < high) {
      int mid = (low + high) / 2;
      mergeSort(arr, low, mid);
      mergeSort(arr, mid + 1, high);
      merge(arr, low, mid, high);
   }
}
int main() {
   int n;
   cout << "Enter number of elements: ";
   cin >> n;
   int arr[100];
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++) {
      cin >> arr[i];
   }
   mergeSort(arr, 0, n - 1);
   cout << "\nSorted Array:\n";
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }    return 0;}
```

Quick Sort

Aim

To sort a list of integers using Quick Sort.

Algorithm

1. Select a pivot element.
2. Partition the array:
    - Elements smaller than pivot move left.
    - Elements greater than pivot move right.
3. Recursively sort left and right subarrays.
4. Repeat until array becomes sorted.

Sample Input

Enter number of elements: 6
Enter elements: 45 12 89 23 1 67

Sample Output

Sorted Array:
1 12 23 45 67 89

Solution

```
#include <iostream>
using namespace std;

// Partition function
int partition(int arr[], int low, int high) {
   int pivot = arr[low];
   int i = low + 1;
   int j = high;
   while (true) {
      while (i <= high && arr[i] <= pivot)
         i++;
      while (arr[j] > pivot)
         j--;
      if (i < j) {
         swap(arr[i], arr[j]);
      }
      else {
         break;
      }
   }
   swap(arr[low], arr[j]);
   return j;
}
void quickSort(int arr[], int low, int high) {// Quick Sort function
   if (low < high) {
      int pivotIndex = partition(arr, low, high);
      quickSort(arr, low, pivotIndex - 1);
      quickSort(arr, pivotIndex + 1, high);
   }
}
int main() {
   int n;
   cout << "Enter number of elements: ";
   cin >> n;
   int arr[100];
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++) {
      cin >> arr[i];
   }

   quickSort(arr, 0, n - 1);
   cout << "\nSorted Array:\n";
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }
   return 0;
}
```

Binary Tree Operations

Aim

To implement binary tree operations using arrays and doubly linked lists.

Binary Tree Operations using Arrays

Aim

To implement binary tree operations using arrays.

Algorithm

1. Store root node at index 0.
2. Left child of node at index i:
   2i+1
3. Right child of node at index i:
   2i+2
4. Insert elements into an array.
5. Display tree elements using traversal.

Operations

- Insertion
- Traversal

Sample Input

Enter number of nodes: 5
Enter elements: 10 20 30 40 50

Sample Output

Binary Tree Created Successfully

Binary Tree Elements:
10 20 30 40 50

Solution

```
#include <iostream>
using namespace std;
class BinaryTree {
   int tree[100];
   int size;
public:
   // Constructor
   BinaryTree() {
      size = 0;
   }
   // Insert node
   void insert(int val) {
      tree[size] = val;
      size++;
   }
   // Display tree
   void display() {
      if (size == 0) {
         cout << "Tree is Empty\n";
         return;
      }
      cout << "Binary Tree Elements:\n";
      for (int i = 0; i < size; i++) {
         cout << tree[i] << " ";
      }
      cout << endl;
   }
};

int main() {
   BinaryTree bt;
   int n, x;
   cout << "Enter number of nodes: ";
   cin >> n;
   cout << "Enter elements: ";
   for (int i = 0; i < n; i++) {
      cin >> x;
      bt.insert(x);
   }
   cout << "\nBinary Tree Created Successfully\n\n";
   bt.display();
   return 0;
}
```

Binary Tree Operations using Doubly Linked List

Aim

To implement binary tree operations using doubly linked list representation.

Algorithm

1. Create node with:
    - left pointer
    - data
    - right pointer
2. Insert nodes level-wise.
3. Perform traversal operations.

Operations

- Insertion
- Traversal

Sample Input

Enter number of nodes: 5
Enter elements: 10 20 30 40 50

Sample Output

Binary Tree Created Successfully

Binary Tree Elements:
10 20 30 40 50

Solution

```
#include <iostream>
#include <queue>
using namespace std;

class Node {

public:
    int data;
    Node* left;
    Node* right;

    // Constructor
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

class BinaryTree {

    Node* root;

public:

    // Constructor
    BinaryTree() {
        root = NULL;
    }

    // Insert node level-wise
    void insert(int val) {

        Node* temp = new Node(val);

        // First node
        if (root == NULL) {
            root = temp;
            return;
        }

        queue<Node*> q;

        q.push(root);

        while (!q.empty()) {

            Node* current = q.front();
            q.pop();

            // Insert left child
```

```
            if (current->left == NULL) {
                current->left = temp;
                return;
            }

            else {
                q.push(current->left);
            }

            // Insert right child
            if (current->right == NULL) {
                current->right = temp;
                return;
            }

            else {
                q.push(current->right);
            }
        }
    }

    // Inorder traversal
    void inorder(Node* root) {

        if (root == NULL)
            return;
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
    }

    // Display function
    void display() {
        cout << "Inorder Traversal:\n";
        inorder(root);
        cout << endl;
    }
};
int main() {
    BinaryTree bt;
    int n, x;
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> x;
        bt.insert(x);
    }
    cout << "\nBinary Tree Created Successfully\n\n";
    bt.display();
    return 0;
}
```

Tree Traversal Methods

Aim

To implement inorder, preorder, postorder, and level order traversal methods for a binary tree.

Algorithms

Inorder Traversal

1. Traverse left subtree.
2. Visit root node.
3. Traverse right subtree.

Traversal Order:Left → Root → Right

Preorder Traversal

1. Visit root node.
2. Traverse left subtree.
3. Traverse right subtree.

Traversal Order:Root → Left → Right

Postorder Traversal

1. Traverse left subtree.
2. Traverse right subtree.
3. Visit root node.

Traversal Order:Left → Right → Root

Level Order Traversal

1. Use queue.
2. Visit nodes level by level from left to right.

Algorithms:

- Inorder: Left, Root, Right.
- Preorder: Root, Left, Right.
- Postorder: Left, Right, Root.
- Level Order: Use queue, level by level.

Sample Input

Enter number of nodes: 7
Enter elements: 10 20 30 40 50 60 70

Sample Output

Inorder Traversal:
40 20 50 10 60 30 70

Preorder Traversal:
10 20 40 50 30 60 70

Postorder Traversal:
40 50 20 60 70 30 10

Level Order Traversal:
10 20 30 40 50 60 70

Solution

```
#include <iostream>
#include <queue>
using namespace std;

class Node {

public:
    int data;
    Node* left;
    Node* right;

    // Constructor
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

class BinaryTree {

public:

    Node* root;

    // Constructor
    BinaryTree() {
        root = NULL;
    }

    // Insert node level-wise
    void insert(int val) {

        Node* temp = new Node(val);

        // First node
        if (root == NULL) {
            root = temp;
            return;
        }

        queue<Node*> q;

        q.push(root);

        while (!q.empty()) {

            Node* current = q.front();
            q.pop();

            // Insert left child
            if (current->left == NULL) {
```

```
            current->left = temp;
            return;
        }

        else {
            q.push(current->left);
        }

        // Insert right child
        if (current->right == NULL) {
            current->right = temp;
            return;
        }

        else {
            q.push(current->right);
        }
    }
}

// Inorder Traversal
void inorder(Node* root) {

    if (root == NULL)
        return;

    inorder(root->left);

    cout << root->data << " ";

    inorder(root->right);
}

// Preorder Traversal
void preorder(Node* root) {

    if (root == NULL)
        return;

    cout << root->data << " ";

    preorder(root->left);

    preorder(root->right);
}

// Postorder Traversal
void postorder(Node* root) {

    if (root == NULL)
        return;

    postorder(root->left);
```

```
        postorder(root->right);

        cout << root->data << " ";
    }

    // Level Order Traversal
    void levelOrder() {

        if (root == NULL)
            return;

        queue<Node*> q;

        q.push(root);

        while (!q.empty()) {

            Node* current = q.front();
            q.pop();

            cout << current->data << " ";

            if (current->left != NULL)
                q.push(current->left);

            if (current->right != NULL)
                q.push(current->right);
        }
    }
};

int main() {
    BinaryTree bt;
    int n, x;
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> x;
        bt.insert(x);
    }
    cout << "\nInorder Traversal:\n";
    bt.inorder(bt.root);
    cout << "\n\nPreorder Traversal:\n";
    bt.preorder(bt.root);
    cout << "\n\nPostorder Traversal:\n";
    bt.postorder(bt.root);
    cout << "\n\nLevel Order Traversal:\n";
    bt.levelOrder();

    return 0;
}
```

Second Minimum Node in Binary Tree

Aim

Write a program to find the second minimum node in a binary tree.

Algorithm

1. Initialize:
    - first_min = INF
    - second_min = INF
2. Traverse the binary tree.
3. For each node:
    - If node value is smaller than first_min:
        - Assign second_min = first_min
        - Assign first_min = node value
    - Else if node value is greater than first_min and smaller than second_min:
        - Assign second_min = node value
4. After traversal:
    - second_min contains the second minimum node.

Sample Input

Enter number of nodes: 5
Enter elements: 10 5 20 3 7

Sample Output

Second Minimum Node = 5

Solution

```
#include <iostream>
#include <queue>
#include <climits>
using namespace std;

class Node {

public:
    int data;
    Node* left;
    Node* right;

    // Constructor
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

class BinaryTree {

public:

    Node* root;

    int first_min;
    int second_min;

    // Constructor
    BinaryTree() {
        root = NULL;

        first_min = INT_MAX;
        second_min = INT_MAX;
    }

    // Insert node level-wise
    void insert(int val) {

        Node* temp = new Node(val);

        // First node
        if (root == NULL) {
            root = temp;
            return;
        }

        queue<Node*> q;
```

```
    q.push(root);

    while (!q.empty()) {

        Node* current = q.front();
        q.pop();

        // Insert left child
        if (current->left == NULL) {
            current->left = temp;
            return;
        }

        else {
            q.push(current->left);
        }

        // Insert right child
        if (current->right == NULL) {
            current->right = temp;
            return;
        }

        else {
            q.push(current->right);
        }
    }
}

// Find minimum and second minimum
void findSecondMinimum(Node* root) {

    if (root == NULL)
        return;

    // Update minimums
    if (root->data < first_min) {

        second_min = first_min;

        first_min = root->data;
    }

    else if (root->data < second_min &&
             root->data != first_min) {

        second_min = root->data;
    }

    findSecondMinimum(root->left);

    findSecondMinimum(root->right);
}
```

```
};

int main() {

    BinaryTree bt;

    int n, x;

    cout << "Enter number of nodes: ";
    cin >> n;

    cout << "Enter elements: ";

    for (int i = 0; i < n; i++) {
        cin >> x;
        bt.insert(x);
    }

    bt.findSecondMinimum(bt.root);

    if (bt.second_min == INT_MAX)
        cout << "Second Minimum Node not found";
    else
        cout << "Second Minimum Node = "
             << bt.second_min;

    return 0;
}
```

Binary Search Tree Operations

Aim

Write a program to insert, delete, and search for a node in a BST.

Algorithm

Insert Operation

1. If tree is empty, create root node.
2. Compare value with current node.
3. If smaller, move to left subtree.
4. If larger, move to right subtree.
5. Repeat until insertion position is found.

Search Operation

1. Compare target value with root node.
2. If equal, element found.
3. If smaller, search left subtree.
4. If larger, search right subtree.

Delete Operation

Three cases:

1. Node is a leaf node → delete directly.
2. Node has one child → replace node with child.
3. Node has two children:
    - Find inorder successor.
    - Replace node with inorder successor.
    - Delete inorder successor.

Sample Input

Enter number of nodes: 7
Insert elements: 50 30 70 20 40 60 80
Search: 40
Delete: 20

Sample Output

40 found in BST
20 deleted successfully

Solution

```
#include <iostream>
using namespace std;

class Node {

public:
    int data;
    Node* left;
    Node* right;

    // Constructor
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

class BST {

public:

    Node* root;

    // Constructor
    BST() {
        root = NULL;
    }

    // Insert node
    Node* insert(Node* root, int val) {

        if (root == NULL) {
            return new Node(val);
        }
        if (val < root->data) {
            root->left = insert(root->left, val);
        }
        else if (val > root->data) {
            root->right = insert(root->right, val);
        }
        return root;
    }

    // Search node
    bool search(Node* root, int key) {

        if (root == NULL)
            return false;
```

```
    if (root->data == key)
        return true;

    if (key < root->data)
        return search(root->left, key);

    return search(root->right, key);
}

// Find minimum node
Node* findMin(Node* root) {

    while (root->left != NULL) {
        root = root->left;
    }

    return root;
}

// Delete node
Node* deleteNode(Node* root, int key) {

    if (root == NULL)
        return root;

    // Search node
    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    }

    else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    }

    else {

        // Case 1: No child
        if (root->left == NULL &&
            root->right == NULL) {

            delete root;
            return NULL;
        }

        // Case 2: One child
        else if (root->left == NULL) {

            Node* temp = root->right;

            delete root;

            return temp;
```

```
            }
            else if (root->right == NULL) {

                Node* temp = root->left;
                delete root;
                return temp;
            }
            // Case 3: Two children
            Node* temp = findMin(root->right);
            root->data = temp->data;
            root->right =
                deleteNode(root->right, temp->data);
        }
        return root;
    }
};

int main() {

    BST tree;
    int n, x;
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Insert elements: ";
    for (int i = 0; i < n; i++) {
        cin >> x;

        tree.root =
            tree.insert(tree.root, x);
    }

    int searchKey;

    cout << "Search: ";
    cin >> searchKey;

    if (tree.search(tree.root, searchKey))
        cout << searchKey
             << " found in BST\n";
    else
        cout << searchKey
             << " not found in BST\n";
    int deleteKey;
    cout << "Delete: ";
    cin >> deleteKey;
    tree.root =
        tree.deleteNode(tree.root, deleteKey);
    cout << deleteKey
         << " deleted successfully";
    return 0;
}
```

Graph Representation using Arrays and Linked Lists

Aim

Write a program to create a graph using arrays (adjacency matrix) and linked lists (adjacency list).

Algorithm

Adjacency Matrix

1. Create a 2D array:
   graph[V][V]
2. Initialize all elements to 0.
3. For every edge (u, v):
   - Set:
     graph[u][v] = 1
     graph[v][u] = 1
     for an undirected graph.
4. Display the adjacency matrix.

Adjacency List

1. Create an array of linked lists.
2. Each index represents a vertex.
3. Insert connected vertices into linked lists.
4. Display adjacency list.

Sample Input

Enter number of vertices: 4
Enter number of edges: 3
Enter edges:0 1
0 2
1 3

Sample Output

Adjacency Matrix:
0 1 1 0
1 0 0 1
1 0 0 0
0 1 0 0
Adjacency List:
0 -> 2 -> 1 -> NULL
1 -> 3 -> 0 -> NULL
2 -> 0 -> NULL
3 -> 1 -> NULL

Solution

```
#include <iostream>
using namespace std;

class Node {

public:
   int data;
   Node* next;

   // Constructor
   Node(int val) {
      data = val;
      next = NULL;
   }
};

class Graph {

   int matrix[20][20];

   Node* list[20];

   int V;

public:

   // Constructor
   Graph(int vertices) {

      V = vertices;

      // Initialize matrix
      for (int i = 0; i < V; i++) {

         for (int j = 0; j < V; j++) {

            matrix[i][j] = 0;
         }
      }

      // Initialize adjacency list
      for (int i = 0; i < V; i++) {
         list[i] = NULL;
      }
   }

   // Insert edge
   void insertEdge(int u, int v) {

      // Adjacency Matrix
      matrix[u][v] = 1;
```

```
    matrix[v][u] = 1;

    // Adjacency List
    Node* temp1 = new Node(v);

    temp1->next = list[u];

    list[u] = temp1;

    Node* temp2 = new Node(u);

    temp2->next = list[v];

    list[v] = temp2;
}

// Display adjacency matrix
void displayMatrix() {

    cout << "\nAdjacency Matrix:\n";

    for (int i = 0; i < V; i++) {

        for (int j = 0; j < V; j++) {

            cout << matrix[i][j] << " ";
        }

        cout << endl;
    }
}

// Display adjacency list
void displayList() {

    cout << "\nAdjacency List:\n";

    for (int i = 0; i < V; i++) {

        cout << i << " -> ";

        Node* temp = list[i];

        while (temp != NULL) {

            cout << temp->data << " -> ";

            temp = temp->next;
        }

        cout << "NULL\n";
    }
}
```

```
};

int main() {

    int V, E;

    cout << "Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;

    cout << "Enter edges:\n";

    for (int i = 0; i < E; i++) {

        int u, v;

        cin >> u >> v;

        g.insertEdge(u, v);
    }

    g.displayMatrix();

    g.displayList();

    return 0;
}
```

BFS and DFS Traversals

Aim

To implement Breadth First Search (BFS) and Depth First Search (DFS) graph traversal methods.

Algorithms

DFS Algorithm (Recursion)

1. Visit current node.
2. Mark node as visited.
3. Recursively visit all unvisited adjacent nodes.

BFS Algorithm (Queue)

1. Visit starting node.
2. Mark node as visited.
3. Insert node into queue.
4. Repeat until queue becomes empty:
    - Remove front node.
    - Visit all unvisited neighbors.
    - Add neighbors into queue.

Sample Input

Enter number of vertices: 5
Enter number of edges: 4
Enter edges:
0 1
0 2
1 3
3 4
Enter starting vertex: 0

Sample Output

BFS Traversal:
0 2 1 3 4

DFS Traversal:
0 2 1 3 4

Solution

```
#include <iostream>
#include <queue>
using namespace std;

class Node {

public:
   int data;
   Node* next;

   // Constructor
   Node(int val) {
      data = val;
      next = NULL;
   }
};

class Graph {

   Node* g[20];

   int vertices;

public:

   // Constructor
   Graph(int V) {

      vertices = V;

      for (int i = 0; i < vertices; i++) {
         g[i] = NULL;
      }
   }

   // Insert edge
   void insertEdge(int u, int v) {

      Node* temp1 = new Node(v);

      temp1->next = g[u];

      g[u] = temp1;

      Node* temp2 = new Node(u);

      temp2->next = g[v];

      g[v] = temp2;
   }
```

```
// DFS Traversal
void DFS(int start, bool visited[]) {

    visited[start] = true;

    cout << start << " ";

    Node* temp = g[start];

    while (temp != NULL) {

        if (!visited[temp->data]) {

            DFS(temp->data, visited);
        }

        temp = temp->next;
    }
}

// BFS Traversal
void BFS(int start) {

    bool visited[20] = {false};

    queue<int> q;

    visited[start] = true;

    q.push(start);

    while (!q.empty()) {

        int current = q.front();

        q.pop();

        cout << current << " ";

        Node* temp = g[current];

        while (temp != NULL) {

            if (!visited[temp->data]) {

                visited[temp->data] = true;

                q.push(temp->data);
            }

            temp = temp->next;
        }
    }
```

```
    }
};

int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;
    Graph g(V);
    cout << "Enter number of edges: ";
    cin >> E;
    cout << "Enter edges:\n";
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.insertEdge(u, v);
    }

    int start;
    cout << "Enter starting vertex: ";
    cin >> start;
    cout << "\nBFS Traversal:\n";
    g.BFS(start);
    bool visited[20] = {false};
    cout << "\n\nDFS Traversal:\n";
    g.DFS(start, visited);

    return 0;
}
```